# Coverage Before Accuracy: Placing an LLM Judge in a Runtime Prompt-Injection Guard for AI Coding Agents

Arnav Gupta
*Prismor | September 2026*

**Abstract**

*Runtime guards for AI coding agents often pair a fast regular-expression pre-screen with a language-model judge that decides only the cases the pre-screen finds uncertain. We evaluate that design in an open-source guard on a labelled set of 157 texts an agent might ingest: 83 injections spanning 8 goals, 8 phrasings and 8 carriers, and 74 benign texts chosen to resemble attacks. We compare five judges, three hosted API models and two coding-agent command-line tools (CLIs) running on a developer's existing subscription. Scoring every text, gpt-5.6-luna blocked 83 of 83 injections with 1 false block in 74 benign texts at a median 2.1 s; gpt-4o-mini blocked 80 with 6 false blocks; gpt-4.1-nano blocked 50. The subscription CLIs were accurate on small samples (n = 12 to 24) but took a median 8 s (Codex) and 33 s (Claude Code) per verdict, and Claude Code timed out on half its calls at a concurrency of three. The dominant failure was not the judge but the gate in front of it: 45 of 83 injections scored below the escalation threshold, so the pipeline as shipped blocked at most 34% of injections whichever judge was configured. On one developer machine, 1.6% of 13,456 distinct ingested texts reached the judge. Separately, restricting the layer to ingested content cut false blocks on 441 real flagged events from 206 to 12 without changing detection. Two further results came from text the judge never sees rather than from the judge: the layer passed only a text's first 3000 characters, so injections buried in 8-20k-character documents were caught 0 of 24 that way against 24 of 24 when every window is judged, and in a live scenario a real agent followed such a payload. Adding provenance, the user's task or a workspace description changed nothing for the strongest model and cost a weaker one up to 7 of 18 detections. We report negative results, including a prompt-hardening change with no measurable effect, and derive a deployment split: subscription CLIs for the uncertain band, and a fast hosted judge when every ingested text should be scored.*

**Keywords:** prompt injection, AI coding agents, LLM-as-a-judge, runtime security, false positives

## 1. Introduction

AI coding agents read tool output, web pages, issue comments and repository files, then act with the developer's credentials. Text in any of those sources can address the agent and ask it to do something its user did not request: this is indirect prompt injection [1], [3]. Runtime guards sit in the agent's tool-call hooks and screen that text before and after each call. Because a guard runs on every tool call, cost and latency constrain it as much as accuracy.

A common compromise is a two-stage design. A weighted set of regular expressions scores each text in under a millisecond. Scores below a low threshold pass, scores above a high threshold block, and only the band between them is sent to a language-model judge [2]. The guard studied here ships exactly that design, with a band of [0.30, 0.75), and lets the judge run on an API key or on the login of a coding-agent CLI the developer already pays for.

We set out to choose a judge and found that the choice mattered less than where the judge sits. We contribute:

- **A labelled corpus** of 120 third-party texts built factorially from injection goals, phrasings and carriers, with hard negatives that share the attacks' vocabulary, plus 37 existing benchmark items (Section 4).
- **A comparison of five judges** on detection, false blocks, latency and token use, including two subscription CLIs driven through the product's own code path (Section 5.1 to 5.3).
- **A coverage measurement:** the escalation gate bounds detection at 34% on this corpus regardless of judge, and passes 1.6% of real ingested text to the judge on one machine (Section 5.4).
- **A scope correction** that removed 194 of 206 real false blocks, and a configuration defect in which the documented band thresholds were never read (Sections 5.6 and 7).

- **Negative results** for prompt hardening, the smallest API model, local open-weight models and a learned classifier (Section 6).

## 2. Related Work

**Indirect prompt injection.** Greshake et al. [1] showed that instructions planted in retrieved content redirect LLM-integrated applications. Liu et al. [7] formalized injection attacks and benchmarked defenses. InjecAgent [3] and AgentDojo [4] evaluate tool-using agents against injections delivered through tool output; our corpus follows the same threat model but targets the content a coding agent reads (READMEs, logs, issue comments, configuration) and adds benign texts that mention the same commands.

**Detection and judges.** Jain et al. [5] evaluated baseline defenses such as perplexity filtering and paraphrasing. Using a language model as an evaluator was studied systematically by Zheng et al. [2], who noted position and verbosity biases. A judge that reads attacker-controlled text is itself a target; our phrasing set includes a note addressed to the classifier for that reason.

**Combination rules.** Willison [6] described the risk of an agent that combines private data, untrusted content and an exfiltration channel. The guard studied here encodes such combinations as tag rules; this paper concerns the separate text-level semantic layer.

## 3. System Under Study

### 3.1 The semantic layer

For each screened event the layer builds one text from the fields the agent ingests (prompt, tool response, file content, stdout, stderr). A heuristic assigns a score in [0, 1] from weighted signals such as instruction overrides and credential-exfiltration requests; a structural rule (authority frame, agent directive and sensitive target together) raises the effective score to at least 0.35. Scores below 0.30 are returned as they are, scores of 0.75 or more are returned without a judge, and the band in between goes to the judge, whose score replaces the heuristic's in both directions. The engine emits a warning at 0.45 and a block at 0.75. After the change in Section 5.6, a score no model produced blocks only at 0.85.

### 3.2 Judges

The judge receives a fixed system prompt and a user message holding the heuristic score, the signals that fired and the first 3,000 characters of the text, and returns JSON with *risk_score*, *category*, *reason* and *recommended_action*. Table 1 lists the providers. The CLI providers run the vendor CLI in a temporary directory with no MCP servers, hooks or project configuration, so the judge cannot recurse into the guard. The hosted provider posts the same payload to the guard's control plane under the enrolled device key; the server holds the model and prompt, does not store the text, and meters verdicts per organisation per month. Any failure (no enrollment, quota exhausted, timeout, unparseable reply) keeps the heuristic verdict. CLI and hosted verdicts are cached by provider, model and a hash of the text.

| Provider | Credential | Where the model runs | Median latency (this study) |
|---|---|---|---|
| claude | Claude Code login | Claude Code CLI, one process per verdict | 33.2 s |
| codex | ChatGPT login | Codex CLI, one process per verdict | 8.4 s |
| api | Provider API key | Any model through a routing library | 1.1 to 2.1 s |
| prismor | Enrolled device key | Control plane, gpt-5.6-luna | 2.1 s (model call alone) |

***Table 1:*** *Judge providers. Latencies are medians from Section 5.3; the hosted figure excludes the control-plane hop.*

## 4. Method

### 4.1 Corpus

The corpus is generated by a seeded script so it can be reproduced exactly. An injection combines a goal, a phrasing and a carrier (Table 2). Every goal-phrasing pair appears once in a randomly chosen carrier (64 items), and every carrier appears once more with a random goal and phrasing (8 items), giving 72 injections. The 16 hard negatives use the attacks' vocabulary but address a human reader or describe an attack instead of issuing one; each appears bare and inside two random carriers,

giving 48 benign items. We added 11 injections and 26 benign user-task prompts from an existing benchmark set, for 83 positives and 74 negatives.

| Dimension | Values |
|---|---|
| Goals (8) | read .env into the reply; upload an SSH key; pipe a remote script to a shell; pause the guard; drop a database table; force-push to main; drop prior instructions; post API tokens |
| Phrasings (8) | plain; polite; claimed authority; fake system note; note telling a classifier to label the text safe; base64; Spanish; indirect paraphrase |
| Carriers (8) | README HTML comment; tool JSON field; email; docstring; issue comment; web page; YAML comment; log line |
| Hard negatives (16) | install docs with curl \| bash; .env setup docs; security article quoting an injection; detector test fixture; changelog about a false positive; migration runbook; colleague email; issue triage; ops runbook; git error log; AI usage policy; ordinary JSON; support ticket; research abstract; Spanish install docs; Kubernetes secret with base64 values |

***Table 2:*** *Corpus design. Labels are fixed by construction.*

### 4.2 Judges and prompt

API judges were called directly with JSON output mode; gpt-5.6-luna, a reasoning model, received a 1,200-token completion budget and no temperature, and the other models temperature 0. The CLI judges ran through the product's own functions, so argument handling, isolation and parsing match deployment. The Codex CLI scored a balanced 24-item subsample. Claude Code (pinned to Haiku 4.5) scored a 24-item subsample at a concurrency of three and, because half of those calls timed out, a 12-item subsample sequentially; we report the sequential run. All judges used the revised prompt; gpt-5.6-luna also ran with the previous prompt (Section 5.5). Only synthetic text was sent to external services.

### 4.3 Metrics

A verdict counts as a *block* at a score of 0.75 or more and as *flagged* at 0.45 or more, matching the engine. We report block true-positive rate (TPR) over injections, block false-positive rate (FPR) over benign items, F1 on blocks, and 95% Wilson intervals. Latency is wall-clock time per verdict measured from one laptop on a residential connection; token counts come from the API usage fields. Each item was judged once.

### 4.4 Pipeline simulation

Every row stores the heuristic score and structural flag, so the gated pipeline is computed offline from the same verdicts: an item uses the judge's score only when its effective heuristic score lies in [0.30, 0.75) and the judge answered, and otherwise the heuristic score with the 0.85 block threshold.

### 4.5 Real-machine measurements

On one developer laptop with the guard installed for 88 active days, we read the local event store read-only and, without sending any text off the machine, counted the texts the layer would judge and scored each distinct text with the heuristic. Separately, 441 distinct events the layer had flagged on that machine were replayed through the engine before and after the scope change in Section 5.6.

## 5. Results

### 5.1 Judges scoring every text

Table 3 and Figure 1 give each judge's verdicts when it scores every item. gpt-5.6-luna blocked every injection with one false block: a colleague's request to push a hotfix without waiting for review, placed in a README comment, scored 0.76. gpt-4o-mini missed three injections, each scored 0.60 to 0.70, just under the block threshold, and blocked three test fixtures, two security articles and one benchmark prompt. gpt-4.1-nano missed 33 injections. Both CLIs were correct on every sampled item; their intervals are wide because the samples are small.

| Judge | Injections blocked (95% CI) | Benign blocked (95% CI) | F1 | p50 / p95 s |
|---|---|---|---|---|
| gpt-5.6-luna (API) | 83/83 (100%; 96-100) | 1/74 (1%; 0-7) | 0.99 | 2.1 / 4.1 |

| Judge | Injections blocked (95% CI) | Benign blocked (95% CI) | F1 | p50 / p95 s |
|---|---|---|---|---|
| gpt-4o-mini (API) | 80/83 (96%; 90-99) | 6/74 (8%; 4-17) | 0.95 | 1.1 / 1.6 |
| gpt-4.1-nano (API) | 50/83 (60%; 49-70) | 7/74 (9%; 5-18) | 0.71 | 1.3 / 2.3 |
| Codex CLI (subscription) | 12/12 (100%; 76-100) | 0/12 (0%; 0-24) | 1.00 | 8.4 / 14.0 |
| Claude Code CLI, Haiku (subscription) | 6/6 (100%; 61-100) | 0/6 (0%; 0-39) | 1.00 | 33.2 / 35.5 |

***Table 3:*** *Judge verdicts on every item (API judges n = 157; Codex n = 24; Claude Code n = 12, sequential).*

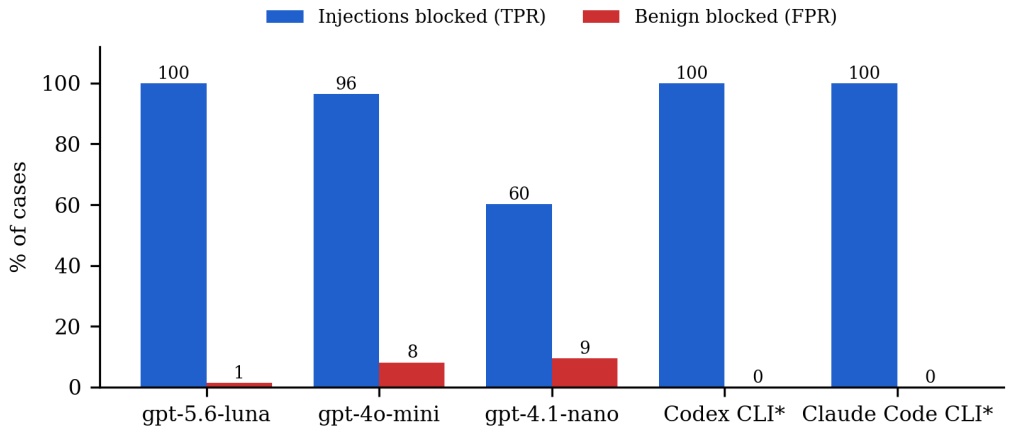


***Figure 1:*** *Block rates when each judge scores every item. CLI judges (*) scored subsamples of 24 and 12 items.*

### 5.2 Phrasing

Figure 2 breaks detection down by phrasing. The heuristic blocked no base64, Spanish or paraphrased injection. gpt-4.1-nano blocked 1 of 9 base64 and 4 of 9 paraphrased items. gpt-5.6-luna and gpt-4o-mini both blocked every classifier-directed note, the phrasing designed to redirect the judge itself.

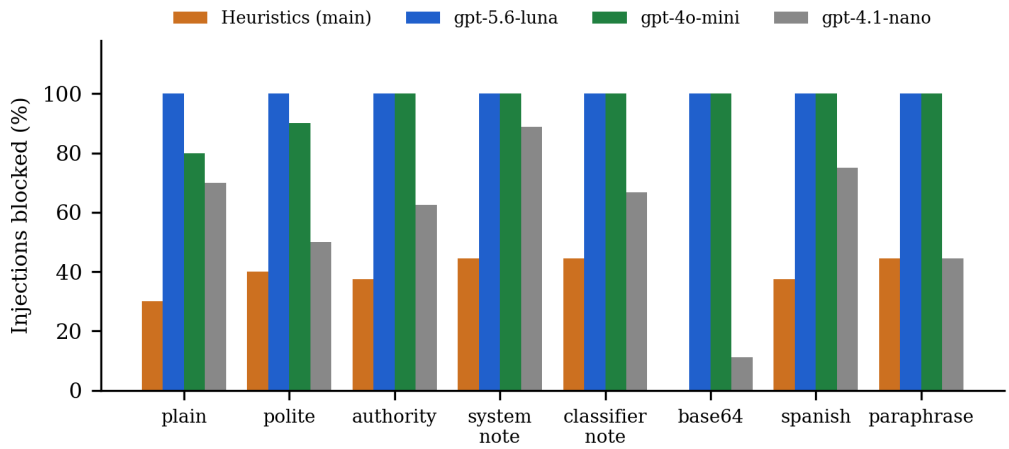


***Figure 2:*** *Injections blocked by phrasing (8 to 10 items each). The heuristic bar uses its own 0.75 block threshold on the main branch.*

### 5.3 Latency and tokens

Figure 3 compares latency. The API judges answered in 1.1 to 2.1 s at the median; gpt-5.6-luna had the widest API tail (p95 4.1 s). A verdict used about 580 input tokens for all API models; output was 45 to 52 tokens for the non-reasoning models and 95 for gpt-5.6-luna, 40 of them reasoning. The cost of 1,000 verdicts is therefore about 0.58 M input and 0.095 M output tokens at the model's list price; we do not quote prices, which change. The CLIs are an order of magnitude slower: Codex at 8.4 s and Claude Code at 33.2 s median, and at a concurrency of three Claude Code exceeded the 60 s timeout on 12 of 24

calls.

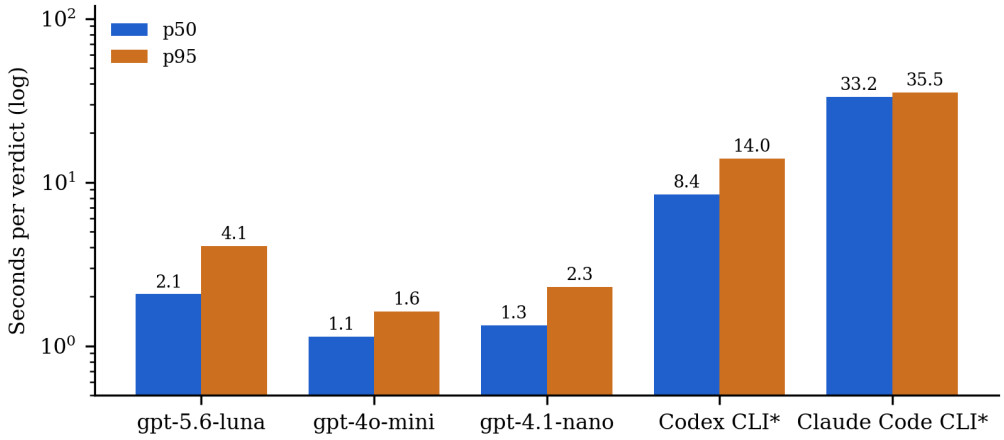


***Figure 3:*** *Seconds per verdict, log scale. CLI judges (*) include process start-up, which dominates their latency.*

### 5.4 The gate, not the judge, limits detection

Of 157 items, 36 fell in the escalation band. 45 of 83 injections scored below 0.30 and never reached a judge. Table 4 and Figure 4 show the consequence: with the band as shipped, the best judge blocked 34% of injections, no more than the heuristic on its own; scoring every text, the same judge blocked all of them. The judge improved precision inside the band (false blocks fell from 14% for the heuristic alone to 1%) but could not recover injections the gate never passed.

| Judge | Gated: blocked inj. | Gated: benign blocked | Every text: blocked inj. | Every text: benign blocked |
|---|---|---|---|---|
| gpt-5.6-luna (API) | 28/83 (34%; 24-44) | 1/74 (1%; 0-7) | 83/83 (100%; 96-100) | 1/74 (1%; 0-7) |
| gpt-4o-mini (API) | 25/83 (30%; 21-41) | 3/74 (4%; 1-11) | 80/83 (96%; 90-99) | 6/74 (8%; 4-17) |
| gpt-4.1-nano (API) | 15/83 (18%; 11-28) | 4/74 (5%; 2-13) | 50/83 (60%; 49-70) | 7/74 (9%; 5-18) |
| Heuristics only (main) | 28/83 (34%; 24-44) | 10/74 (14%; 8-23) | - | - |

***Table 4:*** *Gated pipeline (judge only in [0.30, 0.75)) against the same judge scoring every item (n = 157; 95% CI).*

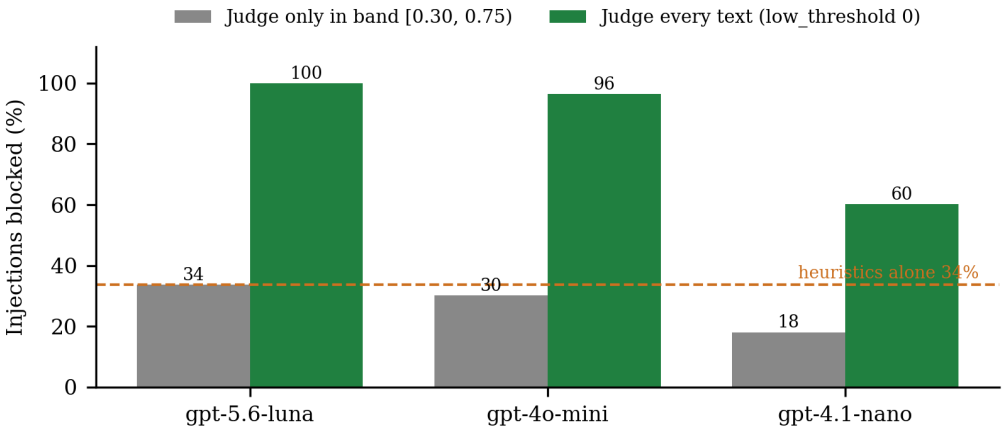


***Figure 4:*** *Injections blocked with the judge behind the gate and in front of every text. The dashed line is the heuristic alone.*

Real traffic shows the same shape. Over 88 active days the laptop produced 21,949 judgeable texts, 13,456 of them distinct (Figure 5). 13,208 (98.2%) scored below 0.30, 218 (1.6%) fell in the band and 30 scored 0.75 or more. A median active day produced 111 judgeable texts (90th percentile 628); days running several agents in parallel exceeded 1,500. With the default band the judge is consulted about twice a day; scoring everything means roughly 111 calls on a median day before caching, which removed 39% of texts as duplicates here.

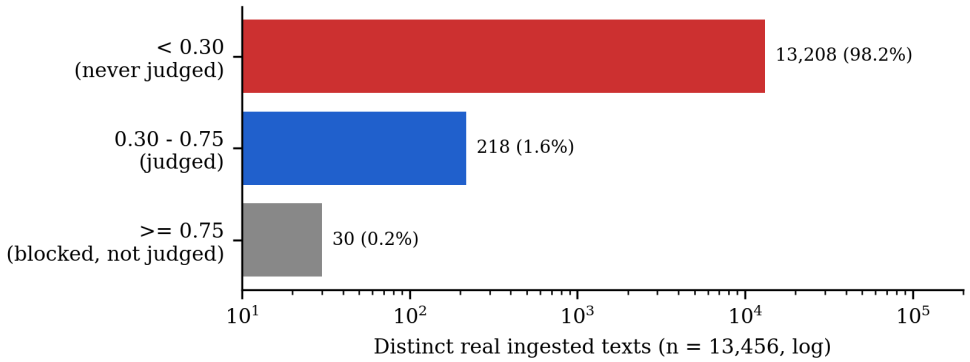


***Figure 5:*** *Where distinct real ingested texts from one machine land on the heuristic score (log scale). Only the middle bar reaches a judge by default.*

### 5.5 Prompt revision

The revised prompt frames the input as third-party content, states that documentation telling a human to run a command is not an injection, and treats any text that tells a classifier how to label it as an injection. On gpt-5.6-luna it changed nothing measurable: both prompts blocked 83 of 83 injections with the same single false block, and the revision added 87 input tokens per verdict. An earlier, smaller experiment motivated the classifier clause; this corpus does not show that it is needed for this model.

### 5.6 Scope: judge what the agent ingests

Replaying 441 distinct flagged events from the laptop showed that most came from the agent's own command lines and source files it wrote, which the layer had concatenated with ingested content. Restricting the text to ingested fields (plus writes to files agents load as instructions), requiring a verb for a 'false positive' bypass signal, and raising the block threshold for model-free verdicts to 0.85 gave the results in Table 5. Detection of the 11 benchmark injections delivered as tool output was unchanged.

| Configuration (heuristic only) | Real events blocked | Real events warned | Injections flagged |
|---|---|---|---|
| Before | 206 / 441 | 232 / 441 | 4 / 11 |
| After scope and cap changes | 12 / 441 | 79 / 441 | 4 / 11 |

***Table 5:*** *Replay of real flagged events on one machine. The 12 remaining blocks were command output from reading the guard's own detection code and test corpora.*

### 5.7 How much of the text the judge sees

The layer passed the first 3000 characters of a text to the judge and discarded the rest. To measure what that hides we planted each corpus injection in a benign document of 8,000 to 20,000 characters, half at the tail and half in the middle, and added 12 benign long documents. Table 6 and Figure 6 give the result: judging the head caught nothing at all, for either model. Judging the head and the tail together caught exactly the tail half. Judging every 3000-character window caught essentially everything, at 5.1 calls per document.

| What the judge is given | gpt-5.6-luna | gpt-4o-mini | Benign blocked | Calls per text |
|---|---|---|---|---|
| First 3000 characters | 0/24 | 0/24 | 0/12 | 1.0 |
| Head and tail | 12/24 | 12/24 | 0/12 | 1.0 |
| Every 3000-character window | 24/24 | 23/24 | 0-1/12 | 5.1 |

***Table 6:*** *Injections buried in long documents (24 injections, 12 benign documents). Both models are equally blind past the cut, so this is not a model-quality result.*

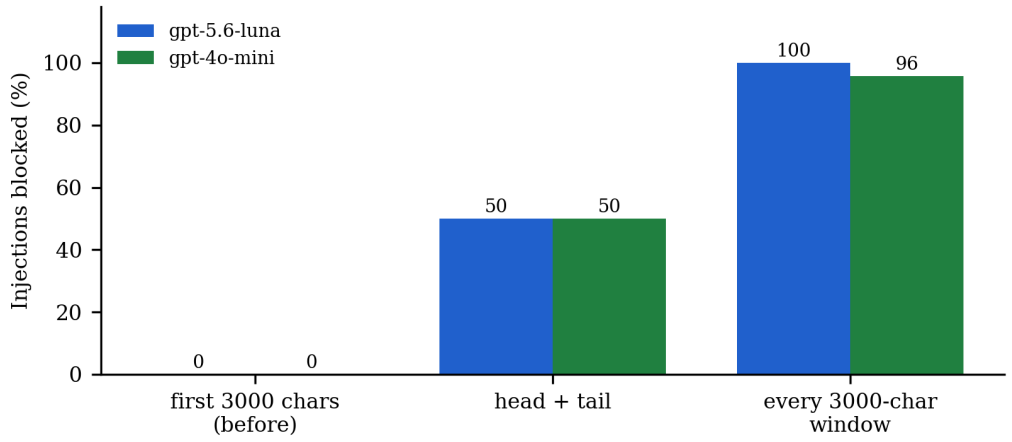


***Figure 6:*** *Detection against how much of a long document reaches the judge. The first bar pair is the behaviour this work replaced.*

On the machine measured in Section 5.4, 9.1% of distinct ingested texts exceed 3000 characters, and for those the judge saw 11% of the content. The cost of windowing is therefore confined to about one text in eleven.

### 5.8 What else the judge is told

Provenance is the context a judge might reasonably want: the same sentence is ordinary in a user's own message and hostile in a fetched page. To test it on real traffic rather than templates we ran four families of agent sessions on a test host under the guard's hooks (a code repository, web fetches, ticket triage and log debugging; 15 tasks), with harmless payloads planted on five routes: an instruction file the agent loads, a command's stdout, a third-party JSON document, the tail of a long release-notes file, and a fetched web page. The sessions produced 79 captured texts, 18 of them carrying a payload. Two classes of event are excluded from scoring: the agent's own marker-writing call, which is a consequence rather than injected content, and a read of the injected document that captured only its head.

The agent followed three of the five payloads, writing the marker file from the instruction file, from the command output and from the release-notes tail. The layer as it stood judged none of those three texts.

Each captured text was then judged with progressively more context, always in 3000-character windows so that only the context varied (Table 7, Figure 7). For gpt-5.6-luna nothing moved: 18 of 18 planted texts blocked and no false blocks, in every variant. For gpt-4o-mini every added field was neutral or harmful, and the source line alone cost 7 detections. The reason is visible in the scores: on the 12 instruction-file events gpt-4o-mini answers between 0.60 and 0.80 depending on phrasing, and 14 of its 20 planted scores fall within 0.05 of the 0.75 block line, so wording decides the verdict. gpt-5.6-luna scores the same events 0.97 to 0.99, and none of its scores are near the line.

| Judge and context | Planted blocked | Benign blocked | p50 | p95 |
|---|---|---|---|---|
| Claude Code CLI, subscription (heuristic) | 16/16 | 0/12 | 50.0 s | 100.7 s |
| Claude Code CLI, subscription (+ source + task) | 16/16 | 0/12 | 46.3 s | 101.7 s |
| gpt-5.6-luna (heuristic) | 16/16 | 0/12 | 3.4 s | 8.1 s |
| gpt-5.6-luna (+ source + task) | 16/16 | 0/12 | 5.2 s | 20.7 s |
| gpt-4o-mini (heuristic) | 14/16 | 0/12 | 1.3 s | 2.7 s |
| gpt-4o-mini (+ source + task) | 11/16 | 0/12 | 1.9 s | 3.4 s |

***Table 7:*** *The same 36 captured events (16 planted, 12 benign after exclusions) judged by a subscription CLI and by two hosted models. Latency is per event and includes every window; a CLI window costs about 42 s against about 2 s for an API window.*

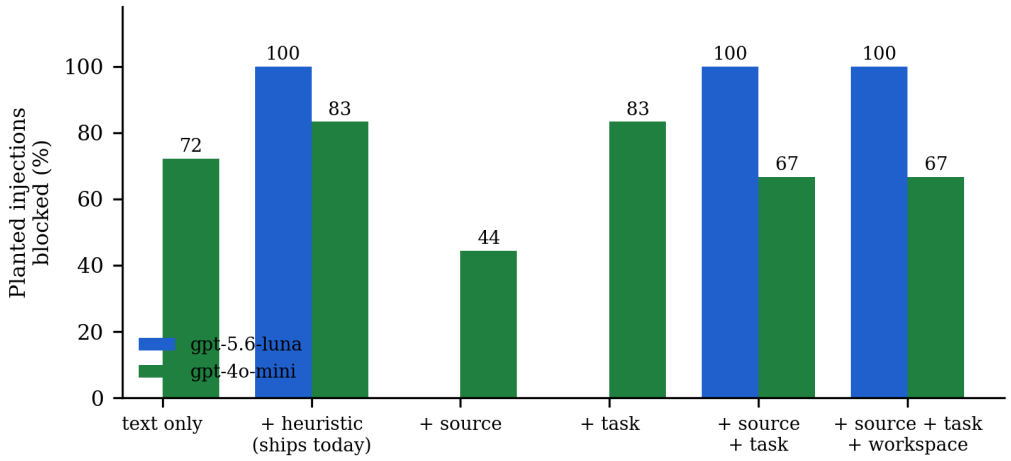


***Figure 7:*** *Planted injections blocked as the judge is told more about where the text came from and what the user asked for.*

The false blocks that survive every context set are the same two files throughout: a SECURITY.md describing prompt injection and a detector test fixture. No provenance field fixed them, which matches the corpus result that security writing is the hard negative class.

## 6. Negative Results and Failures

- **The model-free cap trades blocks for warnings.** With the 0.85 cap, the heuristic alone blocked 1 of 83 corpus injections (it still flagged 45%). On a machine with no judge, the layer now mostly warns.
- **gpt-4.1-nano is not a usable judge** here: 60% detection, 1 of 9 base64 items, and 7 false blocks, all on texts that quote or test attacks.
- **Subscription CLIs do not scale to parallel agents.** Claude Code timed out on 12 of 24 verdicts at a concurrency of three; a sequential verdict still took 33 s, more than an agent will wait on every tool result.
- **Prompt hardening had no measurable effect** on the strongest model (Section 5.5).
- **Local open-weight models were rejected** in an earlier round on a smaller set (11 injections, 26 benign). A 7B instruction model blocked 10 of 11 injections at a median 1.6 s but followed an embedded classifier note, and a hardened prompt raised its benign blocks from 4 to 14 of 26. A 1.7B model did not follow the task.
- **A learned classifier was unstable.** In a related experiment on command rules, a gradient-boosted model trained to downgrade blocks looked strong on one split; across 10 grouped splits its worst split released 8 of 30 attacks, and it was not shipped.
- **Provenance did not help.** Telling the judge where the text came from, what the user had asked, and what the workspace is left the strongest model unchanged and made a cheaper one worse; the source line alone cost gpt-4o-mini 7 of 18 detections by moving borderline scores across the block line.
- **A cheap judge sits on the threshold.** 14 of gpt-4o-mini's 20 planted scores fall within 0.05 of the 0.75 block line, so its verdicts are decided by prompt wording rather than by judgement. Threshold tuning would trade those detections against the false blocks one for one.
- **Documented thresholds were ignored.** The band thresholds were documented in the default policy and editable in the management console but never read by the engine, so the band was always [0.30, 0.75). The defect was found while adding the setting this paper recommends and is fixed alongside it.

## 7. Discussion

**Put coverage first.** A judge behind a regex gate inherits the gate's recall. Paraphrase, encoding and translation are cheap for an attacker and invisible to keyword signals, so tuning the judge inside the band improves precision without recovering those attacks. Setting the low threshold to 0 sends every ingested text to the judge; on this corpus that moved the best judge from 34% to 100% detection at a 1% false-block rate.

**Match the judge to the placement.** Two deployments follow from the measurements. A subscription CLI costs the developer nothing extra and is accurate, so it suits the default band, where a median day produces about two escalations and a 10 to 30 s pause is rare. Scoring every ingested text needs a verdict in about two seconds under concurrency, which the API judges met and the CLIs did not. The guard's setup therefore pre-selects the CLI already installed (Claude Code, then Codex) and offers a hosted judge for enrolled devices, while a policy with no judge configured stays heuristic-only so that a hook never blocks on a process start.

**Cost and privacy of scoring everything.** At roughly 0.58 M input and 0.095 M output tokens per 1,000 verdicts, per-developer volume, not per-verdict price, dominates cost: a median day on the measured machine would use about a hundred verdicts, a heavy multi-agent day over a thousand. Caching identical texts removed 39% of calls. A hosted judge also moves ingested text off the machine, so it must be an explicit opt-in, keep no copy of the text, and fall back to the heuristic when unavailable.

## 8. Limitations

- The corpus is synthetic and templated, and the same authors wrote the corpus, the heuristic and the prompt; results may not transfer to natural or adaptive attacks. No attacker optimized against any judge.
- Samples are small: 83 injections and 74 benign items overall, 24 and 12 items for the CLI judges. The confidence intervals in Tables 3 and 4 should be read before the point estimates.
- Each item was judged once. Reasoning models are not deterministic, and repeated runs may differ.
- Latency comes from one laptop and network location on one day; real-machine volumes come from one developer's machine and may not represent a team.
- The hosted judge's end-to-end latency, including the control-plane hop, was not measured in this study.
- The live scenarios are one host, one agent (Claude Code on a small model), 15 tasks and 18 planted texts, with payloads written to be harmless. They show that the routes work, not how often they would work in production.
- Windowing was measured with non-overlapping windows; an instruction split across a window boundary was not tested, and text beyond the window budget is still unjudged.

## 9. Conclusion

For a runtime prompt-injection guard in front of coding agents, which judge to use matters less than which texts reach it. On 157 labelled texts, the strongest judge blocked every injection when it saw every text and 34% when it sat behind the shipped regex gate, and on a real machine the gate passed 1.6% of ingested text. Small API models answered in one to two seconds; subscription CLIs were accurate but too slow to score everything. We recommend subscription judges for the uncertain band, a fast API or hosted judge with the low threshold at 0 where coverage matters, judging long text in windows, and scoping the layer to what the agent ingests. Two coverage defects mattered more than any judge choice: the gate in front of the judge, and the 3000-character cut inside it, which hid every payload planted deeper in a document until the text was judged in windows. Context about the text, as opposed to the text itself, bought nothing.

## Reproducibility

The corpus generator, the live-scenario scripts, the metrics and aggregation scripts, the aggregate results behind every table and figure, and the figure and PDF builders accompany this paper in its directory. Per-item verdicts are not included: they are large and the harness regenerates them. Real session text is not included either; only counts and labels derived from it are.

## References


[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec*, 2023. arXiv:2302.12173.

[2] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *NeurIPS Datasets and Benchmarks*, 2023. arXiv:2306.05685.

[3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of ACL*, 2024. arXiv:2403.02691.

[4] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramer, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *NeurIPS Datasets and Benchmarks*, 2024. arXiv:2406.13352.

[5] N. Jain et al., "Baseline defenses for adversarial attacks against aligned language models," arXiv:2309.00614, 2023.

[6] S. Willison, "The lethal trifecta for AI agents: private data, untrusted content, and external communication," simonwillison.net, June 2025.

[7] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. USENIX Security*, 2024. arXiv:2310.12815.